
Projet Tutoré

Commande par retour d'état basée observateur
Asservissement en position angulaire & GyroBoy (Segway)



Auteurs : LAROUMAGNE-BONNIOL Caroline & TONYE Alberrique

Encadrante : BARA IULIANA Gabriela

Tuteur : CUVILLON Loïc

21/10/2025 - 13/02/2026

Dépôt Git :

https://github.com/Albernye/Projet_Tutore

Table des matières

1	Introduction et Objectifs du Projet	2
2	Prérequis matériels et logiciels	3
2.1	Configuration logicielle	3
2.2	Configuration matérielle	3
2.2.1	Pour la Partie 1 (Asservissement MCC)	3
2.2.2	Pour la Partie 2 (Segway GyroBoy)	3
3	Architecture du Dépôt et Ressources	4
3.1	Organisation des répertoires	4
3.2	Documentation additionnelle	4
4	Asservissement d'un MCC par observateur d'état	4
4.1	Contexte et problématique (TP 2A)	4
4.2	Synthèse de la loi de commande et de l'observateur	5
4.2.1	Observateur de Luenberger	5
4.2.2	Commande par retour d'état avec action intégrale	5
4.3	Implémentation Simulink	5
4.4	Validation expérimentale et Analyse	5
5	Commande du Segway (GyroBoy)	6
5.1	Objectif : Démonstrateur interactif	6
5.2	Modélisation et Découplage	6
5.2.1	Dynamique Longitudinale (Équilibre et Avance)	6
5.2.2	Dynamique Latérale (Rotation en Lacet)	7
5.3	Logique d'interaction et Capteurs	7
5.4	Implémentation sur l'EV3	7
6	Mode d'emploi et Résultats attendus	8
6.1	Partie 1 : Asservissement du MCC	8
6.1.1	Exécution et Déploiement	8
6.1.2	Résultats attendus	8
6.2	Partie 2 : Commande du Segway (GyroBoy)	8
6.2.1	Exécution et Déploiement	8
6.2.2	Scénario de démonstration (Fête de la Science)	9
6.3	Guide de dépannage	9
7	Conclusion et Perspectives	9
7.1	Bilan du projet	9
7.2	Défis techniques rencontrés	9
7.3	Perspectives d'évolution	10

1 Introduction et Objectifs du Projet

Ce document technique présente les travaux réalisés dans le cadre de notre projet tutoré de dernière année, intitulé « *Commande par retour d'état basée observateur – asservissement en position angulaire et Gyroboy* ». Ce projet s'articule autour de la plateforme robotique LEGO MINDSTORMS EV3 et vise à mettre en pratique des concepts avancés d'automatique (commande dans l'espace d'état, observateur de Luenberger, principe de séparation).

Le projet est divisé en deux objectifs principaux, allant de l'amélioration d'un système didactique existant à la conception d'un démonstrateur interactif complet :

1. **Amélioration de l'asservissement d'un actionneur MCC** : La première phase du projet porte sur l'amélioration d'un TP de 2A concernant l'asservissement en position d'une roue dentée pilotée par un Moteur à Courant Continu (MCC). Dans l'implémentation initiale, la vitesse de rotation était obtenue par une simple dérivation numérique de la position mesurée, ce qui amplifiait fortement le bruit de mesure et les erreurs de quantification. L'objectif ici est d'implémenter un observateur d'état (Luenberger) pour estimer proprement cette vitesse, de l'intégrer dans une commande par retour d'état avec action intégrale, et de valider expérimentalement le principe de séparation.



2. **Enrichissement de la commande du Segway (GyroBoy)** : La seconde phase, à vocation de démonstration pour la *Fête de la Science*, concerne le robot pendule inversé GyroBoy. L'objectif est de concevoir et d'implémenter une commande de rotation autour de l'axe vertical (lacet) afin de permettre au robot de réaliser des « pirouettes ». Cette implémentation s'appuie sur un régulateur LQR (Linear-Quadratic Regulator), et intègre une logique d'interaction avec l'environnement grâce à divers capteurs (couleur, ultrasons, infrarouge).



Objectif de ce document Conformément aux attentes du projet, ce document technique n’est pas un rapport théorique exhaustif, mais un **manuel de réexploitation**. Il a pour vocation de fournir à notre tuteur ainsi qu’aux futurs groupes d’étudiants tous les éléments nécessaires pour comprendre l’architecture de notre code, configurer le matériel, compiler les modèles Simulink, et reproduire nos résultats de manière simple et autonome.

Le document est structuré de la manière suivante : les prérequis matériels et logiciels sont détaillés en **Section 2**, suivis de l’architecture du dépôt Git en **Section 3**. Les développements théoriques et techniques concernant le MCC et le GyroBoy sont respectivement abordés dans les **Sections 4 et 5**. Enfin, la **Section 6** fait office de guide d’utilisation pratique en détaillant la procédure d’exécution et les résultats attendus.

2 Prérequis matériels et logiciels

Afin de pouvoir recompiler les modèles Simulink, déployer le code sur le robot et observer les résultats, une configuration spécifique est requise. Cette section détaille les éléments logiciels et matériels indispensables à la réexploitation du projet.

2.1 Configuration logicielle

L’ensemble du projet a été développé sous l’environnement MATLAB/Simulink. Pour exécuter et modifier les fichiers, le poste de travail doit disposer des éléments suivants :

- **MATLAB et Simulink** : L’installation de base est requise pour ouvrir les scripts (.m) et les modèles (.slx ou .mdl).
- **Simulink Support Package for LEGO MINDSTORMS EV3 Hardware** : Ce module additionnel est indispensable pour interfacier les blocs Simulink avec la brique EV3 et générer le code embarqué. Il peut être installé directement depuis MATLAB via le menu *Add-Ons > Get Hardware Support Packages*.
- **Git** : Un client Git (comme GitHub Desktop ou l’interface en ligne de commande) est recommandé pour cloner le dépôt contenant le code source du projet.

2.2 Configuration matérielle

Le projet s’appuie exclusivement sur les composants de l’édition *Education EV3 core set*.

2.2.1 Pour la Partie 1 (Asservissement MCC)

- **Brique EV3** : Faisant office de microprocesseur pour l’acquisition et la commande.
- **Actionneur MCC** : Un grand servomoteur LEGO EV3 intégrant un encodeur incrémental (mesurant l’angle de rotation), connecté à une roue dentée.
- **Câblage** : Un câble de connexion standard EV3 reliant le moteur à un port de sortie de la brique (ex : Port A).

2.2.2 Pour la Partie 2 (Segway GyroBoy)

Le robot doit être assemblé selon les plans standards du modèle GyroBoy. Les éléments matériels sollicités sont les suivants :

- **Brique EV3** : Montée verticalement au centre du robot.
- **Moteurs (Ports B et C)** : Deux grands servomoteurs EV3, un pour chaque roue, équipés de leurs encodeurs respectifs pour mesurer l’angle parcouru.
- **Capteur Gyroscopique (Port 2)** : Fixé solidement au châssis pour mesurer la vitesse angulaire d’inclinaison (tilt velocity) du robot.

- **Capteur de Couleur (Port 1)** : Attaché à un bras, il est utilisé pour déclencher la logique de commande d'avance et de rotation lors de la détection d'une couleur spécifique.
- **Capteurs d'environnement (Ports 3 ou 4)** : Capteur Infrarouge orientés vers l'avant pour la détection d'obstacles.
- **Connexion PC-EV3** : Le câble USB fourni.

3 Architecture du Dépôt et Ressources

L'intégralité du code source, des modèles de simulation et de la documentation technique est centralisée et versionnée sur notre dépôt Git. L'arborescence a été structurée de manière à séparer clairement les deux phases du projet (l'étude préliminaire sur le moteur seul, puis le développement complet sur le robot).

3.1 Organisation des répertoires

Le dépôt est organisé selon les dossiers principaux suivants :

- **MCC/** : Ce dossier contient les fichiers relatifs à la première phase du projet (amélioration du TP d'asservissement).
 - `observateur.mdl` et `observateur_termes_integral.mdl` : Modèles de simulation des observateurs en boucle ouverte et fermée.
 - `Systeme_reel.mdl` : Modèle final déployé sur l'actionneur physique EV3.
 - `sujetTP_SS_MCC.pdf` : Sujet de TP de référence.
- **Segway/** : Ce dossier constitue le cœur du développement du GyroBoy.
 - `model.m` : **Fichier principal à exécuter en premier.** Il définit les constantes physiques du robot, calcule les matrices d'états (A, B, C, D) et génère les gains de commande (LQR).
 - `model_discret.m` : Script d'étude pour la discrétisation du modèle.
 - `LegoEV3.slx` : Modèle Simulink de base assurant la stabilisation pure par retour d'état.
 - `LegoEV3_poursuite.slx` : Évolution du modèle intégrant la commande en vitesse et la détection d'obstacles.
- **Papier/** : Ce dossier contient la version d'aboutissement du projet.
 - `EV3_Rotation.slx` : Modèle de démonstration complet. Il intègre la stabilisation dynamique, la gestion des couleurs au sol pour le déclenchement des rotations (pirouettes), et la logique d'évitement.

3.2 Documentation additionnelle

À la racine du projet se trouve également un fichier `README.md` récapitulant ces éléments de manière synthétique pour une prise en main rapide directement depuis la plateforme d'hébergement Git.

4 Asservissement d'un MCC par observateur d'état

4.1 Contexte et problématique (TP 2A)

Cette première phase du projet vise à améliorer les travaux pratiques de deuxième année (TP 2A) portant sur l'asservissement en position d'une roue dentée, pilotée par un actionneur de type Moteur à Courant Continu (MCC) du Lego Mindstorms EV3.

Dans l'implémentation originelle du TP, le système est commandé dans l'espace d'état. Cependant, l'encodeur incrémental du moteur ne fournissant que la position angulaire θ , la vitesse de rotation $\dot{\theta}$ était obtenue par une simple dérivation numérique temporelle : $v_k = \frac{\theta_k - \theta_{k-1}}{T_s}$. Cette

approche présente un inconvénient majeur en automatique : la dérivation amplifie drastiquement le bruit de mesure et les erreurs de quantification du capteur. L'objectif de cette section est de s'affranchir de cette dérivation en implémentant un observateur d'état.

4.2 Synthèse de la loi de commande et de l'observateur

4.2.1 Observateur de Luenberger

Pour estimer la vitesse sans recourir à la dérivation, nous avons synthétisé un observateur d'état. En notant le système sous la forme $\dot{x} = Ax + Bu$ et $y = Cx$ (où $x = [\theta, \dot{\theta}]^T$ et $y = \theta$), la dynamique de l'état estimé $\hat{x}$ est régie par :

$$\dot{\hat{x}} = A\hat{x} + Bu + L(y - C\hat{x}) \quad (1)$$

Le gain L (matrice de l'observateur) a été calculé par placement de pôles afin de garantir que l'erreur d'estimation $e = x - \hat{x}$ converge asymptotiquement et rapidement vers zéro.

4.2.2 Commande par retour d'état avec action intégrale

Pour assurer le suivi de la consigne de position sans erreur statique, une action intégrale a été ajoutée à la loi de commande. La commande appliquée au moteur est donc de la forme :

$$u = -K\hat{x} + K_{int} \int (\theta_{ref} - \theta) dt \quad (2)$$

Le calcul indépendant du gain de retour d'état K et du gain d'observation L a permis de vérifier expérimentalement le **principe de séparation**.

4.3 Implémentation Simulink

Pour mener à bien cette étude de manière incrémentale, le développement a été scindé en trois modèles Simulink distincts (disponibles dans le répertoire MCC/ du dépôt Git) :

1. **Système_reel.mdl** : Modèle initial avec retour d'état et action intégrale purs (avec dérivation classique de la position mesurée).
2. **observateur.mdl** : Implémentation de l'observateur fonctionnant en parallèle du système en boucle ouverte, permettant de vérifier la bonne estimation de la vitesse sans impacter la commande.
3. **observateur_terme_integral.mdl** : Fusion des deux approches, où la boucle fermée exploite exclusivement les états estimés par l'observateur.

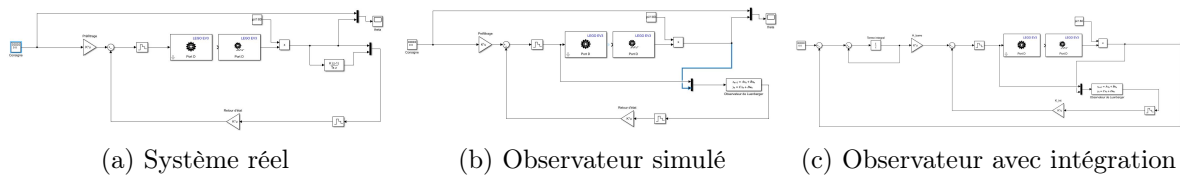


FIGURE 1 – Comparaison

4.4 Validation expérimentale et Analyse

Les différents modèles ont été compilés et déployés sur la brique EV3. Les essais ont été réalisés en salle C003, en utilisant le mode *External* (Monitor & Tune) de Simulink pour observer l'évolution des états en temps réel.

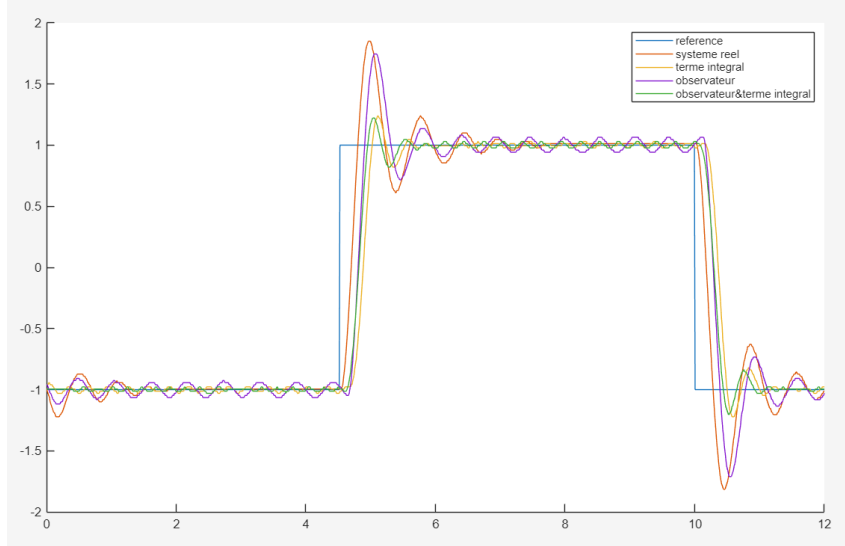


FIGURE 2 – Comparaison de la consigne de vitesse, de la vitesse réelle mesurée, de la vitesse estimée par l’observateur et du terme intégral.

Conclusion sur les performances : L’analyse des signaux via le *Scope* de MATLAB a permis de confirmer que la vitesse estimée par l’observateur ($\hat{v}$) est visuellement beaucoup plus lisse et moins bruitée que celle issue de la dérivation. Cependant, d’un point de vue macroscopique sur les performances de la boucle fermée (temps de réponse, dépassement), **nous n’avons pas constaté de différence de performance significative entre la commande classique et celle basée sur l’observateur**. Ce phénomène s’explique par l’inertie mécanique du système (moteur + roue dentée) qui agit naturellement comme un filtre passe-bas physique, gommant en grande partie les hautes fréquences du bruit de dérivation avant qu’elles ne perturbent le mouvement de la roue.

5 Commande du Segway (GyroBoy)

5.1 Objectif : Démonstrateur interactif

Le second objectif du projet consistait à enrichir la commande du robot Segway (modèle GyroBoy) pour en faire un démonstrateur interactif destiné à la Fête de la Science. Il s’agissait d’ajouter une commande de rotation autour de l’axe vertical (lacet) pour réaliser des « pirouettes », tout en intégrant une logique comportementale basée sur l’environnement.

D’un point de vue mécanique, le corps du robot possède une largeur d’empattement de 8.5 cm, s’étendant à 22 cm en incluant les bras esthétiques.

5.2 Modélisation et Découplage

La dynamique du GyroBoy est complexe et fortement couplée. Notre approche s’appuie sur la modélisation proposée par Hughes *et al.* (2020) [1], qui permet de scinder le système en deux dynamiques distinctes : le mouvement d’avance (Forward) et le mouvement de rotation (Turning).

5.2.1 Dynamique Longitudinale (Équilibre et Avance)

Le vecteur d’état est défini par $x_1 = [\theta, \psi, \dot{\theta}, \dot{\psi}]^T$, où ψ est l’angle d’inclinaison et θ l’angle moyen des roues. La dynamique linéarisée autour de la position d’équilibre vertical est :

$$\dot{x}_1 = A_1 x_1 + B_1 u_1 \quad (3)$$

Pour la stabilisation, un régulateur par placement de pôles a été calculé (gain K_{corr}).

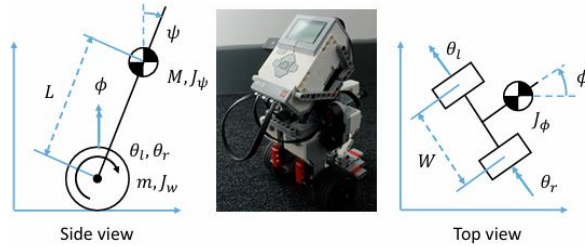
Choix d'asservissement : Lors des simulations, une action intégrale avait été envisagée pour assurer une erreur statique nulle. Cependant, lors du passage sur le robot physique, les imperfections mécaniques (asymétrie, traction des câbles) et le biais du capteur gyroscopique rendaient le maintien strict à $\theta = 0^\circ$ instable. Nous avons donc opté pour un retour d'état simple, complété par un gain de pré-compensation matriciel (S_{corr}) pour asservir la position sans forcer un angle d'équilibre contre-nature.

5.2.2 Dynamique Latérale (Rotation en Lacet)

Pour réaliser la fonction « pirouette », le sous-système de rotation est modélisé avec le vecteur d'état $x_2 = [\phi, \dot{\phi}]^T$, où ϕ représente l'angle de lacet. La dynamique linéarisée s'exprime par :

$$\dot{x}_2 = A_2 x_2 + B_2 u_2 \quad (4)$$

Cette séparation permet d'appliquer un correcteur LQR indépendant pour contrôler l'angle de cap (yaw) sans compromettre l'équilibre longitudinal.



5.3 Logique d'interaction et Capteurs

Pour rendre le robot autonome et interactif, peut recevoir différentes commandes en fonction des mesures effectuées par des capteurs EV3 :

- **Évitement (Infrarouge) :** La détection d'un obstacle rapproché via le capteur infrarouge génère instantanément une consigne de cap $\phi_{ref} = \pm 90^\circ$ sur la boucle latérale, forçant le robot à faire demi-tour ou à esquiver l'obstacle.
- **Pirouette (Couleur) :** Le capteur de couleur attaché à un bras, permet d'effectuer une des quatres actions suivantes, avancer, reculer, tourner à gauche, tourner à droite pendant une durée déterminée (4 secondes), en fonction de la couleur montrée au capteur.

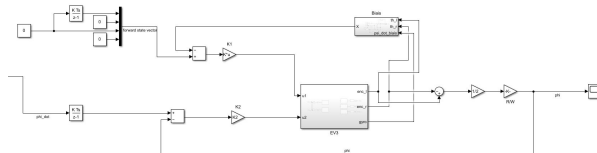
5.4 Implémentation sur l'EV3

La fusion des deux dynamiques s'effectue au niveau de l'affectation matérielle. Le fichier final (EV3_Rotation.slx) combine l'effort longitudinal u_1 et l'effort latéral u_2 pour calculer la tension envoyée aux moteurs gauche (v_l) et droit (v_r) :

$$v_l = u_1 - u_2 \quad (5)$$

$$v_r = u_1 + u_2 \quad (6)$$

Ce mixage permet au GyroBoy de réagir à son environnement tout en compensant en temps réel les perturbations d'inclinaison.



6 Mode d'emploi et Résultats attendus

Conformément aux objectifs du projet, cette section constitue le manuel technique de réexploitation. Elle fournit les instructions pas-à-pas permettant de compiler les scripts MATLAB, de configurer et déployer les modèles Simulink sur la brique EV3, ainsi que les comportements attendus lors des essais physiques.

6.1 Partie 1 : Asservissement du MCC

6.1.1 Exécution et Déploiement

1. **Initialisation de l'espace de travail :** Ouvrez MATLAB et placez-vous dans le répertoire MCC/.
2. Exécutez le script d'initialisation principal :

```
1 >> run AncienTest.m
```

Ce script calcule et charge dans le *Workspace* les matrices d'état discrétisées ($T_e = 10$ ms), les gains de retour d'état avec action intégrale (K_{int} et H_{barre}) et la matrice de l'observateur (L_o).

3. **Ouverture du modèle :** Ouvrez le fichier `observateur_terme_integral.mdl` (ou `Systeme_reel.mdl` pour comparer).
4. **Exécution :** Dans l'onglet *Hardware* de Simulink, vérifiez que l'EV3 est bien connecté (moteur branché sur le port A). Cliquez sur **Monitor & Tune** pour lancer l'exécution en temps réel et remonter les données des capteurs vers le PC.

6.1.2 Résultats attendus

En ouvrant le bloc *Scope* dans Simulink, vous observerez la superposition de deux signaux : la vitesse obtenue par dérivation brute et la vitesse estimée par l'observateur ($\hat{v}$). La courbe de la vitesse estimée doit être significativement plus lisse, démontrant le filtrage efficace du bruit de quantification de l'encodeur. Le suivi de la consigne angulaire doit s'effectuer sans erreur statique.

6.2 Partie 2 : Commande du Segway (GyroBoy)

6.2.1 Exécution et Déploiement

1. **Initialisation :** Placez-vous dans le répertoire `Papier/`.
2. Lancez le script de calcul des modèles couplés (longitudinal et latéral) :

```
1 >> run model.m
2 >> run model_discret.m
```

Vérifiez que les matrices A_1, B_1, A_2, B_2 , le gain de pré-compensation S_{corr} et les gains de commande sont bien chargés.

3. **Ouverture du modèle final :** Naviguez dans le dossier `Papier/` et ouvrez le modèle de démonstration complet `EV3_Rotation.slx`.
4. **Déploiement embarqué :** Posez le robot sur une surface plane. Dans l'onglet *Hardware*, cliquez sur **Build, Deploy & Start**. Le code C sera généré et le programme s'exécutera de manière autonome sur la brique.

6.2.2 Scénario de démonstration (Fête de la Science)

Le modèle `EV3_Rotation.slx` intègre une machine à états gérant plusieurs comportements autonomes. Voici le déroulement attendu :

- **Calibration** : Afin d'être dans le cadre de la linéarisation du modèle, il faut faire bien attention à lâcher le robot dans une position verticale.
- **Stabilisation** : Le robot est sensé se stabiliser autour du point d'équilibre et ne pas se déplacer dans l'espace.
- **Évitement (Capteur Infrarouge)** : Si un obstacle s'approche, une consigne de lacet $\phi_{ref} = \pm 90^\circ$ est envoyée à la boucle de rotation. Le robot esquivé l'obstacle en tournant sur lui-même.
- **Pirouette (Capteur de Couleur)** : Présentez une pastille de couleur spécifique devant le capteur de couleur. Le GyroBoy effectuera alors l'action indiquée par la couleur montrée (avancer, reculer, tourner à gauche, tourner à droite) pendant une durée de 4 secondes avant de reprendre son équilibre standard.

6.3 Guide de dépannage

En cas de comportement erratique, voici les vérifications d'usage pour le GyroBoy :

- **Le robot chute violemment dès le démarrage** : Les câbles des moteurs (Ports B et C) ou leur affectation logicielle sont probablement inversés. Cela transforme la contre-réaction stabilisante en une boucle à réaction positive divergente.
- **Le robot dérive en accélérant dans une direction** : La calibration initiale a échoué (mouvement lors de l'allumage). L'offset du gyroscope est faux, forçant le robot à corriger un angle nul. Redémarrez le programme en assurant une immobilité totale après avoir rebranché le câble du gyroscope dans une position verticale.
- **Erreur lors de la compilation ("Undefined variable")** : Assurez-vous d'avoir exécuté `model.m` dans le *Base Workspace* de MATLAB avant de lancer le *Build* dans Simulink.

7 Conclusion et Perspectives

7.1 Bilan du projet

Ce projet tutoré a permis de mettre en œuvre, de valider et de confronter à la réalité physique des concepts fondamentaux de l'automatique moderne sur la plateforme didactique LEGO MINDSTORMS EV3.

Dans la première phase consacrée à l'actionneur MCC, l'implémentation d'un observateur d'état a prouvé son efficacité pour reconstruire un signal de vitesse propre sans recourir à la dérivation numérique, validant ainsi expérimentalement le principe de séparation. Bien que l'inertie mécanique du système ait lissé l'impact direct sur les performances macroscopiques de la boucle fermée, l'intérêt qualitatif sur le signal de commande a été démontré.

La seconde phase a abouti à la création d'un démonstrateur autonome et interactif pour la *Fête de la Science*. En s'appuyant sur le découplage des dynamiques d'avance et de rotation [1], il a été possible de stabiliser le pendule inversé GyroBoy tout en lui intégrant des comportements réactifs (esquive d'obstacles via infrarouge, et exécution de pirouettes déclenchées par la couleur).

7.2 Défis techniques rencontrés

Le principal enseignement de ce projet réside dans les contraintes imposées par le passage de la théorie à la pratique :

- **Écarts de modélisation et Action intégrale** : L'intégration de l'erreur d'inclinaison (θ) s'est révélée inadaptée sur le robot physique. En raison des asymétries de construction et de la traction des câbles, le point d'équilibre naturel n'est jamais exactement à 0° .

Forcer l'annulation de cette erreur statique rendait le système instable. Le choix final s'est donc porté sur un gain de pré-compensation (S_{corr}) associé au retour d'état.

- **Bruit et dérive des capteurs :** Les capteurs MEMS *low-cost* du kit LEGO présentent des biais significatifs. La dérive du gyroscope a exigé la mise en place d'une routine stricte de calibration au démarrage, étape indispensable pour éviter une dérive rectiligne de la commande.

7.3 Perspectives d'évolution

Le code et l'architecture documentés dans ce dépôt offrent une base solide pour de futures évolutions. Les groupes d'étudiants suivants pourront explorer les pistes suivantes :

1. **Optimisation du filtre de Kalman (LQG complet) :** Exploiter les matrices de covariance du bruit des moteurs (W) et des capteurs (V) pour affiner le gain optimal de l'observateur d'état, en approfondissant l'approche stochastique proposée par Hughes *et al.* [1].
2. **Planification de trajectoire :** Étendre l'asservissement en poursuite actuel pour permettre au Segway de suivre un chemin complexe, par exemple via un algorithme de suivi de ligne continu exploitant le capteur de couleur à haute fréquence.
3. **Télémétrie et réglage *On-the-fly* :** Développer un tableau de bord (*Dashboard*) sous Simulink exploitant les blocs UDP Wi-Fi pour ajuster les gains matriciels (K et L) en direct pendant que le robot s'équilibre, sans nécessiter de recompilation complète.

En conclusion, la brique EV3 s'est avérée être un formidable outil pédagogique. Elle confronte l'étudiant aux réalités du prototypage matériel (quantification, bruit, asymétries) tout en restant pleinement accessible via l'environnement MATLAB/Simulink.

Références

- [1] T. H. Hughes, G. H. Willetts, and J. A. Kryczka, “Lqg controller for the lego mindstorms ev3 gyroboy segway robot,” *IFAC-PapersOnLine*, vol. 53, no. 2, pp. 17282–17287, 2020.
- [2] J. A. Kryczka, T. H. Hughes, and G. H. Willetts, “Legomindstormsev3.” <https://github.com/jakubk28/LegoMindstormsEV3>, 2020. Dépôt du code source associé à l’article IFAC.
- [3] L. C. TONYE A., “Projet_tutore.” https://github.com/Albernye/Projet_Tutore, 2025-2026. Dépôt du code source de notre projet.